Task 1

Table Structure (Products)

```
CREATE TABLE products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(50),
   category VARCHAR(30),
   price DECIMAL(10,2),
   stock INT,
   brand VARCHAR(30)
);
```

Sample Data (20 Records)

```
INSERT INTO products VALUES
(1, 'iPhone 13', 'Mobile', 70000, 10, 'Apple'),
(2, 'Galaxy S21', 'Mobile', 60000, 15, 'Samsung'),
(3, 'Redmi Note 11', 'Mobile', 20000, 25, 'Xiaomi'),
(4, 'MacBook Air', 'Laptop', 90000, 5, 'Apple'),
(5, 'Dell Inspiron', 'Laptop', 55000, 8, 'Dell'),
(6, 'HP Pavilion', 'Laptop', 60000, 7, 'HP'),
(7, 'Sony Headphones', 'Accessories', 8000, 20, 'Sony'),
(8, 'Boat Earbuds', 'Accessories', 2000, 50, 'Boat'),
(9, 'iPad', 'Tablet', 50000, 12, 'Apple'),
(10, 'Samsung Tab', 'Tablet', 30000, 18, 'Samsung'),
(11, 'Asus Laptop', 'Laptop', 65000, 6, 'Asus'),
(12, 'OnePlus 11', 'Mobile', 55000, 14, 'OnePlus'),
(13, 'Realme Narzo', 'Mobile', 15000, 30, 'Realme'),
(14, 'Lenovo ThinkPad', 'Laptop', 75000, 4, 'Lenovo'),
(15, 'JBL Speaker', 'Accessories', 10000, 22, 'JBL'),
(16, 'Mi Pad', 'Tablet', 25000, 16, 'Xiaomi'),
(17, 'AirPods', 'Accessories', 15000, 9, 'Apple'),
(18, 'Oppo F21', 'Mobile', 22000, 19, 'Oppo'),
(19, 'Vivo V27', 'Mobile', 28000, 17, 'Vivo'),
(20, 'Noise Watch', 'Accessories', 5000, 40, 'Noise');
```

SQL Practice Questions (20)

◆ GROUP BY (Basic → Intermediate)

1. Find total number of products in each category

Ans. select category, count(*) as total_products from products

Group by category;

2. Find total stock available for each category

Ans. select category, SUM(stock) AS total_stock

from products

group by category;

3. Find average price of products in each brand

Ans. select brand,avg(price) as average_price
From products
Group by brand;

4. Count how many products each brand has
Ans. select brand,COUNT(*) AS pdt_count
From products
Group by brand;

5. Find maximum price in each category

Ans. select category,MAX(price) AS maximum_price

From products

Group by category;

◆ GROUP BY + ORDER BY

6. Find total stock per brand and sort it in descending order

Ans. select brand,sum(stock) as total_stock

From products

```
Group by brand

Order by total_stock  desc;
```

7. Find average price per category and display highest first

Ans.
```
select category,avg(price) as average_price
From products
Group by category
Order by  average_price desc;
```

8. Count products per category and sort by count (low to high)

Ans.
```
select category,count(*) as total_pdts
From products
Group by category
Order by  total_pdts asc;
```

◆ HAVING (Filtering Groups)

9. Show categories having more than 3 products

Ans.
```
select category,count(*) as total_pdts

from products

group by category

having count(*)>3;
```

10. Show brands where total stock is greater than 40

Ans.
```
select brand,sum(stock) as total_stk

from products

group by brand

having sum(stock) >40;
```

11. Find categories where average price is less than 30,000

Ans.
```
select category,avg(price) as average_price
from products
group by category
having avg(price) <30000;
```

12. Show brands having more than 2 products

Ans.
```
select brand,count(*) as total_pdts
from products
group by brand
having total_pdts>2;
```

13. Find categories where maximum price is above 70,000

Ans.
```
select category,MAX(price) as maximum_price
from products
group by category
having MAX(price)>70000;
```

◆ GROUP BY + HAVING + ORDER BY (Important)

14. Find brands with total stock > 30 and sort by stock descending

Ans.
```
select brand,sum(stock) as total_stk
from products
```

```
group by brand
having total_stk>30
order by total_stk desc;
```

15. Show categories with avg price > 40,000 and sort highest first

Ans.
```
select category,avg(price) as average_price
from products
group by category
having avg(price)>40000
order by average_price desc;
```

16. Count products per brand, show only brands with count ≥ 2, sort by count desc

Ans.
```
select brand,COUNT(*) as total_pdts
from products
group by brand
having COUNT(*)>=2
order by total_pdts desc;
```

◆ LIMIT (Top Results)

17. Get top 5 most expensive products

Ans.
```
select * from products
order by price DESC
LIMIT 5;
```

18. Get 3 cheapest products

Ans. select * from products

order by price ASC

LIMIT 3;

◆ LIMIT + OFFSET (Pagination Concept)

19. Skip first 5 highest priced products and get next 5

Ans. select * from products

Order by price DESC

LIMIT 5 OFFSET 5;

20. Get 4 products starting from the 3rd cheapest product

Ans. select * from products

order by price ASC

LIMIT 4 OFFSET 2;

Task 2

🛒 Amazon / Flipkart Style SQL Scenarios

Table

```
Create table employees(
  emp_id INT,
  name VARCHAR(50),
  age INT,
  department VARCHAR(50),
  location VARCHAR(50),
  join_year INT
);
```

1. 🏆 Top Departments by Hiring

Find the top 2 departments with the highest number of employees.

Ans.
```
SELECT department, COUNT(*) AS total_employees
FROM employees
GROUP BY department
ORDER BY total_employees DESC
LIMIT 2;
```

---

2. ⊘ Ignore First Top Department

Find the 2nd and 3rd highest hiring departments.

Ans. SELECT department, COUNT(*) AS total_employees

FROM employees

GROUP BY department

ORDER BY total_employees DESC

LIMIT 2 OFFSET 1;

---

3. 📍 Cities with High Workforce

Find locations that have more than 2 employees.

Ans. SELECT location, COUNT(*) AS total_employees

FROM employees

GROUP BY location

HAVING COUNT(*) > 2;

---

4. 🎯 Experienced Departments

Find departments where the average age of employees is greater than 28.

Ans. SELECT department, AVG(age) AS avg_age

FROM employees

GROUP BY department

HAVING AVG(age) > 28;

---

5. □ Oldest Workforce Departments

Find the top 3 departments with the highest maximum age.

Ans. SELECT department, MAX(age) AS max_age

FROM employees

GROUP BY department

ORDER BY max_age DESC

LIMIT 3;

---

6. 📅 Recent Hiring Trend

Find the top 2 years with the highest number of employees joined.

Ans. SELECT join_year, COUNT(*) AS total_hired

FROM employees

GROUP BY join_year

ORDER BY total_hired DESC

LIMIT 2;

---

7. 🏢 Balanced Teams

Find departments having between 3 and 6 employees.

Ans. SELECT department, COUNT(*) AS total_employees

FROM employees

GROUP BY department

HAVING COUNT(*) BETWEEN 3 AND 6;

---

8. 📶 Mid-Level Hiring (Pagination)

Find the next 3 departments after the top 2 departments based on employee count.

Ans. SELECT department, COUNT(*) AS total_employees

```
FROM employees
GROUP BY department
ORDER BY total_employees DESC
LIMIT 3 OFFSET 2;
```

---

9. 🏁 Top Locations by Avg Age

Find the top 3 locations with the highest average age of employees.

Ans.
```
SELECT location, AVG(age) AS avg_age
FROM employees
GROUP BY location
ORDER BY avg_age DESC
LIMIT 3;
```

---

10. 🔍 Filter + Group + Sort (Real Interview)

Find departments with more than 2 employees where employees are older than 25, and show the top 2 departments based on count.

Ans.
```
SELECT department, COUNT(*) AS total_employees
FROM employees
WHERE age > 25
GROUP BY department
HAVING COUNT(*) > 2
ORDER BY total_employees DESC
LIMIT 2;
```